

Retrieval-Augmented Generation (RAG) Systems ? Lecture Notes

PAGE 1 ? INTRODUCTION TO RAG

Retrieval-Augmented Generation (RAG) is an architecture that combines information retrieval with large language model (LLM) text generation. Instead of relying only on the knowledge stored inside a model's parameters, a RAG system first retrieves relevant documents from an external knowledge base and then passes them to the LLM as context for generating an answer.

Why RAG matters:

1. Reduces hallucinations ? the model grounds its answers in retrieved evidence.
2. Keeps knowledge fresh ? documents can be updated without retraining the model.
3. Enables private/domain data ? company documents, lecture slides, or research papers can be searched without fine-tuning.
4. Provides citations ? retrieved chunks can be shown as sources to the user.

A typical RAG pipeline has two phases: an offline indexing phase and an online query phase. During indexing, documents are split into chunks and converted into vector embeddings. During querying, the user question is embedded, similar chunks are retrieved, and the LLM generates an answer conditioned on those chunks.

PAGE 2 ? DOCUMENT CHUNKING

Chunking is the process of splitting long documents into smaller passages before embedding them. Chunk size is a critical hyperparameter:

- Too large: chunks contain mixed topics, embeddings become "blurry", and irrelevant text wastes the LLM's context window.
- Too small: chunks lose surrounding context and retrieval returns fragments that are hard to interpret.

Common strategies include fixed-size chunking (e.g. 500-1000 characters with 100-150 character overlap), sentence-aware chunking that avoids cutting sentences in half, and structure-aware chunking that respects headings, paragraphs, pages, or slides. Overlap between consecutive chunks helps preserve context at chunk boundaries so that information spanning two chunks is not lost.

For lecture slides and PDFs, page-based chunking is popular because it preserves the page number as metadata, which allows page-range filtering and precise citations like "see page 12".

An embedding model maps text to a dense numeric vector such that semantically similar texts are close together in vector space. For example, the model `glove-embed-text` produces 768-dimensional vectors.

Similarity between two vectors is usually measured with:

- Cosine similarity: the cosine of the angle between two vectors. Value ranges from -1 to 1, where 1 means identical direction.
- Cosine distance: defined as 1 minus cosine similarity; smaller is more similar.
- Euclidean (L2) distance: straight-line distance between vector endpoints.
- Dot product: combines direction and magnitude; common when vectors are normalized.

During retrieval, the query is embedded with the same model used for documents. A nearest-neighbor search then finds the top-K chunks with the smallest distance to the query vector. K is typically between 4 and 10.

PAGE 4 ? VECTOR DATABASES AND PGVECTOR

A vector database stores embeddings and supports fast nearest-neighbor search. Examples include Pinecone, Weaviate, Milvus, Qdrant, and Chroma. Relational databases can also store vectors using extensions.

pgvector is a PostgreSQL extension that adds a VECTOR column type plus distance operators: <-> for L2 distance, <=> for cosine distance, and <#> for negative inner product. Because pgvector lives inside PostgreSQL, the application can combine vector similarity with ordinary SQL filters, for example:

```
SELECT content FROM document_chunks
WHERE material_id = '...'
ORDER BY embedding <=> '[0.1, 0.2, ...]'
LIMIT 8;
```

For large tables, approximate indexes such as IVFFlat or HNSW dramatically speed up search at a small cost in recall. Supabase ships pgvector out of the box, which makes it convenient for student projects and prototypes.

PAGE 5 ? THE GENERATION STEP AND PROMPT ASSEMBLY

After retrieval, the top-K chunks are assembled into a context block that is inserted into the LLM prompt. A typical prompt template looks like:

"Answer the question using ONLY the context below.

Context: {retrieved_chunks}

Question: {user_question}"

Good practice for the generation step:

1. Order chunks by relevance and include source labels (filename, page number).
2. Enforce a maximum context budget (e.g. 12,000 characters) so the prompt fits the model's context window.
3. Instruct the model to say "I don't know" when the context does not contain the answer, instead of guessing.
4. Ask the model to cite which source supports each claim.

If retrieval returns nothing useful, systems often fall back to a non-RAG mode or widen the search, for example by removing strict filters such as page ranges.

PAGE 6 ? EVALUATION AND COMMON FAILURE MODES

RAG systems are evaluated on two axes: retrieval quality and answer quality.

Retrieval metrics:

- Recall@K: fraction of queries where a relevant chunk appears in the top K.
- Precision@K: fraction of retrieved chunks that are actually relevant.
- MRR (Mean Reciprocal Rank): how high the first relevant chunk ranks.

Answer metrics:

- Faithfulness: does the answer only state facts supported by retrieved context?
- Answer relevance: does the answer actually address the question?
- Citation accuracy: do cited pages really contain the claimed information?

Common failure modes:

1. Chunking errors ? relevant information split across chunks and never fully retrieved.
2. Embedding mismatch ? using different models for indexing and querying.
3. Lost in the middle ? LLMs pay less attention to facts buried in the middle of a long context.
4. Stale index ? documents updated but old embeddings never re-indexed.

A practical tip: log every retrieval (query, top-K chunks, distances) so that failures can be debugged by inspecting what the LLM actually saw.